Cairo University - Faculty of Engineering

EECE Department

4th Year – Second Semester

Neural Networks ELC4028

Spring Semester of Academic Year: 2025 / 2026

Project

GitHub Repository: ELC4028-Neural-Networks

Website: github.io/ELC4028-Neural-Networks/

All the material are uploaded on the website

Name	ID	Sec
ايريني حسني ماهر رمزي	9220175	1
محمد خالد محمد شتا	9203230	3
محب ميخائيل حشمت ميخائيل	9220638	3

Submitted to

Dr. Mohsen Rashwan

Dr. Mohammed Abdelghany

Date of Submission: May 2026

Contents

1 Introduction and Aim of the Work **2**
1.1 Why Word Embeddings? 2
1.2 Project Objectives 2

2 Methodology — Steps Followed **2**
2.1 Step 1 — Topic Selection & Resource Collection 3
2.2 Step 2 — Content Mastery 3
2.3 Step 3 — Arabic Presentation 3
2.4 Step 4 — Arabic Word Embeddings Interactive Demo 3
2.5 Step 5 — Website Generation & Integration 4
2.6 Step 6 — Content Enrichment 4
2.7 Step 7 — Arabic Translation 4
2.8 Step 8 — MCQ Bank Generation 4
2.9 Step 9 — Deployment 5

3 Project Deliverables **5**

4 AI Tools Used **5**
4.1 How Claude Was Used 5
4.2 Human Oversight and Validation 6

5 Website Overview **6**
5.1 Website Sections 6

6 Challenges Faced and How They Were Managed **7**
6.1 Challenge 1 — Arabic Technical Terminology 7
6.2 Challenge 2 — Integrating the Arabic Demo into the Website 7
6.3 Challenge 3 — GitHub Pages Deployment Configuration 7

7 Conclusions **7**

1 Introduction and Aim of the Work

This report documents the design, development, and delivery of a project for the ELC4028 Neural Networks course. The project was assigned as a team exercise in which each group selects one concept from Artificial Neural Networks (ANN), develops a deep understanding of it, and communicates that understanding clearly and interactively using AI-assisted tools.

Our team selected Word Embeddings as our concept — a foundational technique in Natural Language Processing (NLP) that enables neural networks to work meaningfully with text by representing words as dense, learned vectors in a continuous geometric space.

1.1 Why Word Embeddings?

Word Embeddings sit at the intersection of representation learning, linear algebra, and language understanding. They are an ideal teaching concept because they:

- Have a clear, visual intuition — similar words cluster in vector space
- Span multiple algorithms (Word2Vec, GloVe, FastText, ELMo) that illustrate an evolution of ideas
- Connect naturally to broader deep learning concepts: embedding layers, transfer learning, and backpropagation
- Lend themselves to striking interactive demonstrations (vector arithmetic, nearest-neighbor search)

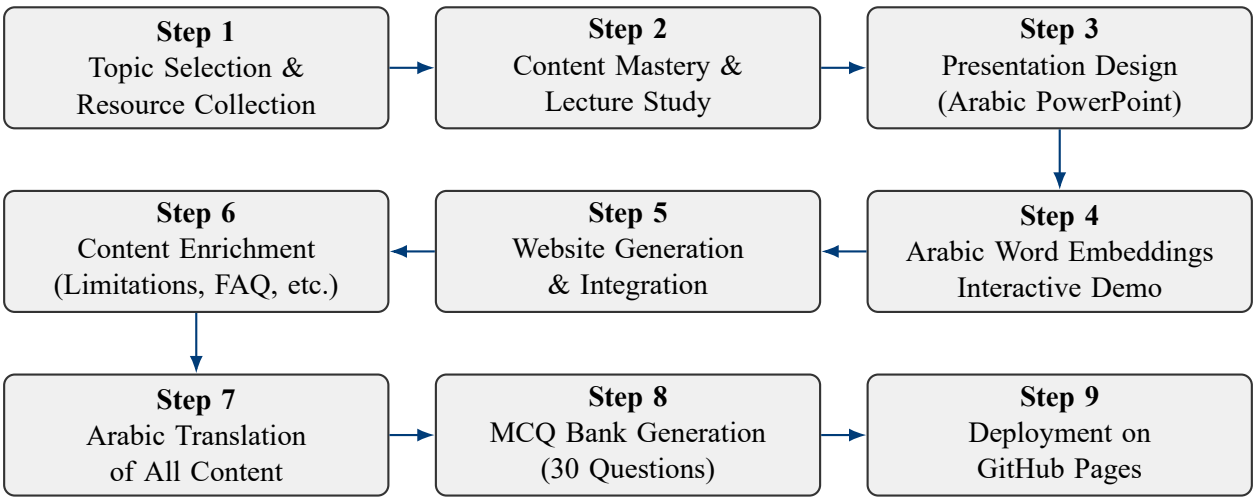
1.2 Project Objectives

The project aimed to achieve the following:

- Produce a visually clear and structured Arabic-language presentation explaining the concept of word embeddings
- Build an interactive bilingual website that teaches word embeddings with embedded demos, quizzes, comparisons, and limitations sections
- Create an Arabic 2D/3D word-space visualization demo showing how semantically related Arabic words cluster together in embedding space
- Utilize AI assistants to build interactive components that visualize and compare the different embedding architectures (Word2Vec, GloVe, FastText, ELMo)
- Design a 30-question MCQ bank covering all major aspects of the topic
- Document the AI tools used and demonstrate genuine understanding of all generated material

2 Methodology — Steps Followed

The project followed a structured pipeline from concept selection through deployment. The following block diagram summarizes the workflow:



Each step is described in detail below.

2.1 Step 1 — Topic Selection & Resource Collection

The team reviewed the list of ANN concepts available and agreed on Word Embeddings. The primary resource was the lecture material by Dr. Ahmad El Sallab (163 slides), supplemented by external references including the Stanford CS224n course notes, Jay Alammar's illustrated blog on Word2Vec, and the original Mikolov et al. (2013) Word2Vec paper.

2.2 Step 2 — Content Mastery

Before any generation began, team members studied the lecture slides and references thoroughly. This ensured the team could evaluate, correct, and explain all AI-generated material rather than accepting it blindly. The 163-slide lecture was uploaded to Claude and an 8-page structured summary was generated in both English and Arabic as a study aid.

2.3 Step 3 — Arabic Presentation

The team designed a PowerPoint presentation in Arabic covering the full arc of the topic: the motivation for embeddings, one-hot encoding limitations, Word2Vec (CBOW, Skip-Gram, SGNS), GloVe, FastText, ELMo, and comparisons. The slides were structured for clarity and visual flow, with diagrams, equations, and worked examples. The presentation was later uploaded to the website.

2.4 Step 4 — Arabic Word Embeddings Interactive Demo

Inspired by Word2Vec visualization tools, the team asked Claude to build an Arabic-language interactive demo that visualizes dimensionality-reduced word vectors in a simulated embedding space. The demo offers a comprehensive suite of features to interact with the data:

- **3D Visualization**: Users can rotate, zoom, and explore a 3D scatter plot of word vectors.
- **Dimensionality Reduction**: Toggling between t-SNE and Raw projection views.
- **Search and Similarity**: A search function that highlights a word and lists its nearest neighbors with cosine similarity percentages.
- **Semantic Operations (Vector Arithmetic)**: Users can perform classic analogies (e.g., ملك − رجل + امرأة = ملكة) or input custom operations.

- **Similarity Map**: A dedicated heatmap to visually analyze the cosine similarity matrix across words.
- **Concepts Visualization**: Grouping words by semantic categories (e.g., royalty, countries, emotions) using a filterable legend.
- **Architecture and Pipeline**: Educational panels explaining CBOW, Skip-gram, and the Arabic NLP pipeline.

Disclaimer: To ensure smooth browser performance without requiring a backend server, the demo uses a representative sample of mock vector data rather than a full, massive 300-dimensional model (like AraVec). This sample data is carefully crafted to accurately reflect the semantic clustering and vector arithmetic properties of real word embeddings.

This interactive demo was then seamlessly integrated into the project website.

2.5 Step 5 — Website Generation & Integration

A starter website was generated using Lovable (an AI-powered web application generator). This gave the team a well-structured React-based site with the core layout, learn/demo/quiz sections. GitHub Copilot was then used to integrate the Arabic demo into the website and to wire together the various components. The site was built to be bilingual (Arabic and English).

2.6 Step 6 — Content Enrichment

Antigravity and GitHub Copilot were used to add deeper content sections beyond the starter site, including:

- Limitations section — covering OOV, static vectors, and the evolution toward contextual models
- Comparison section — Word vs. Sentence Embeddings, Word2Vec vs. GloVe
- Interactive FAQ / misconceptions section with category filters
- Analogies section with interactive vector arithmetic demonstrations
- Methods timeline comparing Word2Vec, GloVe, FastText, and ELMo interactively

2.7 Step 7 — Arabic Translation

Claude was used to translate the full 8-page English summary into Modern Standard Arabic (الفصحى المعاصرة). This served as the basis for Arabic content across the website. The translation was reviewed by team members to ensure technical accuracy and natural phrasing, with special care taken over technical terms that are standardized in Arabic academic literature (e.g., تضمين for embedding, حاصل الضرب الداخلي for dot product).

2.8 Step 8 — MCQ Bank Generation

Claude was prompted to generate 30 multiple-choice questions based on the lecture content and the 8-page summary. Each question was reviewed and verified by the team against the source material. Questions were designed to cover: limitations, embedding matrix mechanics, Word2Vec variants, GloVe objective, FastText, ELMo, transfer learning rules, and vector arithmetic properties. The MCQ bank was integrated into the website as an interactive quiz with randomized question selection.

2.9 Step 9 — Deployment

GitHub Copilot (agent mode) was used to configure and deploy the website to GitHub Pages. The deployed website is publicly accessible at: https://erinih.github.io/ELC4028-Neural-Networks/

3 Project Deliverables

The project produced all four required deliverables, plus an additional website deliverable:

Deliverable 1	Structured bilingual presentation in Arabic — uploaded to the project website
Deliverable 2	Interactive Arabic Word Embeddings 2D/3D demo — embedded on the website
Deliverable 3	30-question MCQ bank with correct answers — integrated as an interactive quiz
Deliverable 4	This project report — documenting aim, methodology, AI tool usage, challenges, and conclusions
Bonus	Full project website deployed on GitHub Pages — https://erinih.github.io/ELC4028-Neural-Networks/

4 AI Tools Used

The project made extensive use of AI tools as assistants — not substitutes. Every AI-generated output was reviewed, corrected where necessary, and understood by the team before being included in the final deliverables.

AI Tool	Role in Project	Outcome / Deliverable
Claude (Anthropic)	Generated 8-page English summary from 163-slide lecture; translated full summary into Modern Standard Arabic; generated 30 MCQ questions; built the Arabic interactive word embeddings demo	Summary (EN/AR), Arabic demo, MCQ bank
Lovable	AI web app generator used to scaffold the starter React website with core structure, layout, and sections	Starter website (React/TypeScript)
GitHub Copilot	Integrated Arabic demo into website; added content sections (limitations, comparison, FAQ, analogies); handled GitHub Pages deployment configuration	Enriched website, deployment
Antigravity	Assisted with uploading and displaying presentation slides on the website interactively; additional UI/content edits	Interactive slides on website

4.1 How Claude Was Used

Claude served as the primary content-generation assistant. Specifically:

- The resources was uploaded and Claude was asked to produce a structured 8-page summary organized by concept (not just slide-by-slide), requiring synthesis and reorganization of ideas.
- The English summary was then passed back to Claude with a prompt to translate into MSA, with specific instructions to use academic terminology rather than colloquial terms.
- For the MCQ bank, Claude was prompted with the summary and asked to generate 30 questions of varying difficulty, covering all major topics, with clearly marked correct answers and explanations. All questions were then verified against the source material.
- For the Arabic demo, Claude was described the goal (imitate Word2Vec visualization but for Arabic words, using dimensionality reduction to show semantic clusters) and generated the interactive React component.
- To compare architectures, Claude was utilized to generate interactive visual components that contrast the different embedding models (Word2Vec, GloVe, FastText, ELMo) side-by-side.

4.2 Human Oversight and Validation

All AI-generated content was subjected to the following validation process:

- Technical accuracy: Every MCQ answer and explanation was cross-checked against the lecture material
- Arabic translation: Technical terms were verified against standard Arabic NLP and mathematics textbooks
- Demo behavior: The Arabic word space demo was tested to confirm that semantically related words (ملك/ملكة, فرنسا/باريس) correctly clustered together
- Website content: All sections were read and edited by team members for clarity, accuracy, and pedagogical value

5 Website Overview

The project website (https://erinih.github.io/ELC4028-Neural-Networks/) serves as the central hub for all deliverables. It is a fully interactive, bilingual (Arabic/English) educational platform covering word embeddings from first principles to modern methods.

5.1 Website Sections

Learn Introduces the core problem (why word indices fail), limitations, and what embeddings solve — with expandable concept cards

Interactive Demo 2D word space visualization — click any word to see its nearest neighbors and cosine similarity scores

Analogies Interactive vector arithmetic demos (king − man + woman ≈ queen) with step-by-step reveal

Methods Side-by-side interactive comparison of Word2Vec, GloVe, FastText, and ELMo — click any method to see details

Limitations Covers OOV failure cases and the evolution to sub-word (FastText) and contextual (ELMo) models

FAQ Filterable misconceptions section covering 8 common misunderstandings, organized by topic

Arabic Demo Dedicated Arabic word embeddings demo featuring 3D visualization (with zoom and rotate), search, dimensionality reduction, similarity map, semantic operations, and concepts visualization. (Note: Uses sample data to simulate a real model).

Quiz 30-question MCQ bank with randomized 5-question practice mode, scoring, and explanations

Presentation Full Arabic PowerPoint slides displayed interactively on the website

6 Challenges Faced and How They Were Managed

6.1 Challenge 1 — Arabic Technical Terminology

Translating technical NLP concepts into Arabic presented challenges where no single standard translation exists (e.g., 'embedding', 'dot product', 'softmax'). The team cross-referenced multiple Arabic NLP publications and academic textbooks to identify the most widely accepted terms. Where a direct translation was ambiguous, the English term was kept alongside the Arabic in parentheses.

6.2 Challenge 2 — Integrating the Arabic Demo into the Website

The Arabic word embeddings demo was initially built as a standalone React component. Integrating it into the existing website structure (which used a different routing and state management pattern) required careful coordination using Lovable and GitHub Copilot. Some component props and event handlers needed to be refactored, and the embedding coordinate data had to be loaded as a static JSON asset rather than being fetched dynamically.

6.3 Challenge 3 — GitHub Pages Deployment Configuration

The React website required a specific base-path configuration for GitHub Pages (since it is served from a repository subdirectory, not the root). GitHub Copilot agent mode was used to identify and fix the `vite.config.ts` base URL setting and update the routing accordingly. The deployment was then verified by checking all internal links and assets after going live.

7 Conclusions

This project demonstrated that AI tools, when used thoughtfully as assistants rather than substitutes, can dramatically accelerate the production of high-quality educational material. The team was able to:

- Transform a 163-slide technical lecture into a fully interactive bilingual website in a fraction of the time it would take manually
- Produce an Arabic-language interactive visualization that makes an abstract mathematical concept (vector spaces) concrete and explorable
- Leverage AI to successfully visualize and compare different complex embedding architectures in an accessible, interactive format

- Generate and verify a 30-question assessment bank that covers the topic comprehensively
- Deploy a publicly accessible website that goes beyond the minimum deliverables by including limitations, comparisons, FAQs, and analogies sections

The most important lesson was that effective use of AI tools requires genuine domain understanding. The team's familiarity with the lecture material was essential for evaluating, correcting, and guiding AI outputs. Without that understanding, the AI-generated content would have been accepted uncritically, including the errors that were found and fixed.

Word Embeddings proved to be an excellent choice of concept: visually compelling, mathematically rich, and directly connected to the broader deep learning curriculum. The interactive website format allowed the team to present the concept in a way that is self-paced, engaging, and reusable as a learning resource well beyond the scope of this course.